

Sistema Fintech de Préstamos Digitales para Bolivia

Software Design Document (SDD)

Parte III

Arquitectura Técnica, DDD e Infraestructura

Versión 1.0

Tabla de Contenido

1. Objetivos de Arquitectura
 2. Principios de Diseño
 3. Arquitectura General
 4. Dominios (DDD)
 5. Arquitectura Modular
 6. Arquitectura de Eventos
 7. APIs
 8. Modelo de Datos
 9. Infraestructura
 10. Inteligencia Artificial
 11. Seguridad
 12. Observabilidad
 13. Escalabilidad
 14. Tecnologías
-

1. Objetivos de Arquitectura

La arquitectura deberá cumplir los siguientes objetivos:

- Alta disponibilidad.
- Escalabilidad horizontal.
- Alta cohesión.
- Bajo acoplamiento.
- Fácil mantenimiento.
- Evolución por módulos.

- Multiempresa (Multi-Tenant).
- API First.
- Cloud Native.
- Preparada para IA.

La plataforma debe permitir crecer desde una startup con pocos clientes hasta una solución utilizada por múltiples entidades financieras.

2. Principios de Diseño

El proyecto seguirá los siguientes principios:

Domain Driven Design (DDD)

Cada dominio representará un proceso de negocio independiente.

Clean Architecture

Separación entre:

- Dominio
 - Aplicación
 - Infraestructura
 - Interfaces
-

SOLID

Todo el código deberá seguir SOLID.

Event Driven Architecture

Los módulos se comunicarán mediante eventos.

Ejemplo

SolicitudCreada

↓

Evaluación

↓

PréstamoAprobado

↓

ContratoGenerado

↓

DesembolsoRealizado

↓

PagoRegistrado

API First

Todo será consumido mediante APIs.

Incluso el Frontend.

Modular Monolith (Fase inicial)

En lugar de iniciar con microservicios.

Se utilizará un **Modular Monolith**.

Ventajas:

- menor complejidad
- menos infraestructura
- menor costo
- más rápido de desarrollar

Cuando el negocio crezca podrá migrarse a microservicios.

3. Arquitectura General





4. Dominios (DDD)

Identity

Responsabilidad

- autenticación
- autorización
- usuarios
- roles
- permisos

Customer

Responsabilidad

Clientes.

Incluye

- perfil
- referencias
- historial
- documentos
- score

Lead

Prospectos.

Responsabilidad

Clientes aún no convertidos.

Loan

Toda la lógica del préstamo.

Incluye

- simulación
- aprobación
- desembolso
- cronograma

Application

Solicitudes.

Todo inicia aquí.

Document

Documentos.

Subida.

OCR.

Versiones.

Validaciones.

Collection

Cobranza.

Recordatorios.

Gestores.

Promesas.

Mora.

Payment

Pagos.

Caja.

Transferencias.

QR.

Comprobantes.

Notification

Centraliza

Email

SMS

WhatsApp

Push

AI

Servicios inteligentes.

OCR.

Clasificación.

Predicción.

Resúmenes.

Reporting

BI.

KPIs.

Dashboard.

Configuration

Toda configuración dinámica.

Audit

Historial completo.

5. Arquitectura Modular

Cada dominio será un módulo independiente.

```
src
identity/
customer/
lead/
application/
loan/
payment/
collection/
notification/
```

```
document/  
  
ai/  
  
audit/  
  
report/  
  
configuration/  
  
shared/
```

Cada módulo tendrá

```
domain/  
  
application/  
  
infrastructure/  
  
presentation/
```

6. Flujo Arquitectónico

```
Landing  
  
↓  
  
WhatsApp  
  
↓  
  
Bot  
  
↓  
  
API  
  
↓  
  
Application Module  
  
↓
```

Customer Module



Document Module



AI Module



Rule Engine



Loan Module



Notification Module

7. Comunicación por Eventos

Ejemplo

Solicitud creada



ApplicationCreated

Eventos



CustomerUpdated

OCRStarted

AIStarted

ScoreCalculated

EvaluationRequested



LoanApproved



ContractGenerated



MoneyDisbursed



PaymentReceived

8. APIs

Todas REST inicialmente.

Posteriormente GraphQL.

Ejemplo

POST

/api/applications

Crear solicitud.

GET

/api/customers

Lista clientes.

POST

/api/payments

Registrar pago.

POST

/api/documents/upload

Subir documento.

POST

/api/whatsapp/webhook

Mensajes.

POST

/api/ai/analyze

Análisis IA.

9. Modelo de Datos

Principales entidades

User
Role
Permission
Lead
Customer
Reference
Application
Loan
Installment
Payment
Document
Notification
Audit
Campaign
Conversation
AIResult
Rule
RuleExecution
Attachment
Comment

Relaciones

Customer

↓

Applications



Loan



Installments



Payments

Documentos

Customer



Documents



OCR



Verification

10. Base de Datos

PostgreSQL

Motivos

- ACID
- JSONB
- índices
- particiones
- estabilidad

Redis

Uso

- cache
 - sesiones
 - rate limit
 - colas
-

S3

Almacenará

- documentos
 - contratos
 - selfies
 - comprobantes
-

Elastic/OpenSearch (Fase futura)

Búsquedas.

11. Motor de Reglas

No estará programado.

Será configurable.

Ejemplo

```
IF
edad < 21
THEN
Solicitar Garante
```

```
IF
score > 90
THEN
```

Aprobar

IF

cliente mora

THEN

Rechazar

Cada regla tendrá

- prioridad
- condiciones
- acciones
- vigencia
- historial
- auditoría

12. Inteligencia Artificial

La IA será un servicio transversal.

No un módulo aislado.

Funciones

OCR

Leer documentos.

LLM

Conversaciones.

Resumen

Resumir expedientes.

Riesgo

Analizar probabilidad.

Fraude

Detectar inconsistencias.

Recomendaciones

Ayudar al analista.

Clasificación

Clientes.

Cobranza

Determinar prioridad.

13. WhatsApp

Será un canal más.

No contendrá lógica.

WhatsApp

↓

Webhook

↓

Notification Module

↓

Conversation Module

↓

Así será posible agregar posteriormente

Telegram

Messenger

Web Chat

App móvil

Sin modificar la lógica.

14. Seguridad

JWT

Refresh Tokens

MFA

HTTPS

TLS

Hash Argon2

Rate Limit

CSRF

CORS

Helmet

Validaciones

Sanitización

Logs

Backups

Documentos

Encriptados.

Contratos

Firmados digitalmente.

Toda acción

Auditada.

15. Observabilidad

Logs

Métricas

Trazabilidad

Errores

Performance

Alertas

Dashboard

Cada solicitud tendrá

TraceID

para seguir todo el flujo.

16. Escalabilidad

Primera etapa

1 servidor

↓

Segunda

API

+

Redis

+

Postgres

↓

Tercera

Balanceador

↓

Múltiples APIs

↓

Redis Cluster

↓

Postgres Replica

↓

S3

↓

Queue

↓

Cuarta

Microservicios.

17. Infraestructura AWS

Servicios recomendados

CloudFront

ALB

ECS Fargate

RDS PostgreSQL

Elasticache Redis

S3

SES

SNS

Secrets Manager

CloudWatch

IAM

WAF

AWS Backup

EventBridge

SQS

18. Tecnologías

Frontend

Next.js

TypeScript

TailwindCSS

Shadcn UI

TanStack Query

React Hook Form

Zod

Backend

NestJS

TypeScript

Prisma ORM

BullMQ

Passport

JWT

OpenAPI

Base de Datos

PostgreSQL

Redis

S3

IA

OpenAI

Claude

Gemini

OCR

Embeddings

RAG (para futuras funciones de soporte interno)

DevOps

Docker

GitHub Actions

Terraform (fase avanzada)

AWS

Monitoreo

Grafana

Prometheus

Loki

CloudWatch

19. Decisiones Arquitectónicas

¿Por qué no Microservicios?

Porque aumentan la complejidad desde el inicio.

Se utilizará un Modular Monolith preparado para dividirse posteriormente.

¿Por qué PostgreSQL?

Mayor estabilidad.

Excelente soporte transaccional.

Ideal para sistemas financieros.

¿Por qué NestJS?

- Modular.
 - DDD friendly.
 - Excelente integración con TypeScript.
 - Escalable.
 - Amplia comunidad.
-

¿Por qué Next.js?

- Excelente SEO para la landing.
 - Renderizado híbrido (SSR/SSG).
 - Alto rendimiento.
 - Ideal para el portal del cliente y panel administrativo.
-

Conclusión

La arquitectura propuesta prioriza la simplicidad inicial sin sacrificar la capacidad de crecimiento. Mediante un **Modular Monolith basado en DDD y Clean Architecture**, la plataforma podrá evolucionar de forma controlada hacia microservicios si el volumen de usuarios y transacciones lo requiere.

Los dominios estarán desacoplados mediante eventos, permitiendo incorporar nuevas capacidades —como nuevos productos financieros, canales de atención o servicios de IA— sin afectar el núcleo del sistema.

Esta arquitectura servirá como base para un ecosistema fintech moderno, preparado para operar inicialmente en Bolivia y expandirse posteriormente a otros mercados de Latinoamérica.